Universidad Mariano Gálvez de Guatemala

Ingeniería en Sistemas

Algoritmos

Ing. Emanuel Mazariegos



Documentación del Proyecto Final

Integrantes

1) Francisco Javier Gonzalez Velásquez	**0903-26-3434**
2) David Alejandro Mazariegos Mérida	**0903-26-2636**
3) Melissa Alexia Bautista Miranda	**0903-26-21788**

Plan Fin de Semana

2do semestre

Sección “A”

30 de octubre del 2026

1. **PARTE TÉCNICA**

Descripción del sistema: El sistema es una aplicación ejecutada en un lenguaje de programación Python, pensada para la administración y control de inventario comercial. El software ofrece un espacio de trabajo estructurado que abarca distintos puntos, desde la restricción de acceso a partir de autenticación de usuarios, análisis de datos y ordenamiento algorítmico manual.

A. **Metodología de creación:**

- **Integrante 1 Francisco Javier: Núcleo y control:**
- **Integrante 2 David Alejandro: Módulo CRUD y Validación de Entradas:**
- **Integrante 3 Melissa Alexia: Analítica, ordenamiento y presentación:** Programar funciones de analítica de datos (totales, promedios, máximos, mínimos), implementación

Integrante 1

Estructura Principal del Código y Ciclos Base

Sección 1: Importación de Librerías y Definición de Variables Iniciales

En esta parte inicial se establecen las listas fundamentales que emulan la base de datos del inventario (codigo_p, nombre_P, precio_P, cantidad_P). Igualmente se establecen las variables de control de seguridad: las credenciales autorizadas del administrador (usuario_est y contraseña_est) así como las variables numéricas control (número máximo de intentos permitidos) y control2 (contador de fracasos en los intentos), que aseguran un acceso al sistema regulado.

```
codigo_p = ["P001", "P002", "P003","P004"]
nombre_P = ["Monitor", "Teclado", "Mouse", "Impresora"]
precio_P = [300, 70, 40, 800]
cantidad_P = [5, 15, 15, 5]

# --- Variables de Control de Acceso ---
usuario_est = "admin"
contraseña_est = "Acces2610"
control = 3
control2 = 0
```

Sección 2: Pantalla de Bienvenida y Ciclo Principal de Autenticación

Esta sección está relacionada con la interfaz visual de entrada y el núcleo del control de acceso. Se presenta un encabezado con estilo y se comienza un ciclo while que depende de la bandera inicio y de que los intentos fallidos no excedan el límite fijado. Se piden las credenciales a lo largo del ciclo. El sistema envía un mensaje de éxito y permite el acceso al menú principal si los datos introducidos corresponden exactamente a las credenciales del administrador. En caso contrario, redirige al bloque de control de errores.

```
inicio = True
while inicio ==True and control2 < control:
    usuario = input("Ingrese su usuario: ")
    contraseña = input("Ingrese su contraseña: ")

    if usuario == usuario_est and contraseña == contraseña_est:
        print("Credenciales válidas. Acceso concedido.")
```

Sección 3: Despliegue del Menú Principal de Opciones

Después de que se haya verificado la identidad del usuario, el núcleo operativo del sistema de gestión se activa mediante un segundo ciclo while True. Se presenta aquí un menú interactivo en formato desplegable que lista las 7 funciones principales. El sistema utiliza input() para capturar la elección del usuario y la transforma en un número entero, lo que la deja lista para su correspondiente enrutamiento lógico.

```
while True:
    print("========================================")
    print("            Menú de Opciones")
    print("========================================")
    print("Inventario_____________________(1)")
    print("Agregar Producto_______________(2)")
    print("Busca Producto_________________(3)")
    print("Eliminar Producto______________(4)")
    print("Cálculos Estadísticos__________(5)")
    print("Ordenar Productos______________(6)")
    print("Salir del Sistema______________(7)")
    print("")

    try:
        opcion=int(input("¿Qué operación desea realizar? (1-7):  "))
        print("========================================")
```

Sección 4: Estructura de Enrutamiento y Control de Excepciones

Este bloque asegura la fortaleza y la experiencia del usuario frente a entradas equivocadas. Si el usuario introduce caracteres especiales o letras en vez de un número, usa la estructura try-except para detectar excepciones de tipo ValueError. Un conjunto de condiciones (if-elif-else) conduce el flujo hacia la opción elegida si la entrada es válida. La opción 7, que realiza una secuencia de cierre seguro, sobresale en particular. Esta cambia el estado de la bandera principal a false y termina el ciclo para finalizar la ejecución de manera ordenada.

```
try:
    opcion=int(input("¿Qué operación desea realizar? (1-7):  "))
    print("========================================")


    if opcion == 1:
        print("\nSeleccionó la opción de Inventario ")
        print("\nEl inventario actual es: \n")
        listar_inventario()

        pass
        print("Presione la tecla Enter para regresar al menú principal")

    elif opcion == 2:
        print("\nSeleccionó la opción de Agregar Producto.")
        registrar_producto()

        pass
        input("Presione la tecla Enter para regresar al menú principal")
```

```
    elif opcion == 7:
        print("Cerrando el sistema")
        print("¡Hasta pronto!")
        inicio = False
        break

    else:
        print("Error, ingrese una opción válida entre 1 y 7")
except ValueError:
            print("Error, debe ingresar un valor numérico válido.")
            print("Ingrese un valor entre 1 y 7")
            continue
```

Sección 5: Gestión de Intentos Fallidos y Bloqueo de Acceso

La seguridad frente a credenciales incorrectas se trata en esta última sección. El bloque else aumenta el contador control2 e imprime un mensaje de advertencia que muestra cuántos intentos quedan si el usuario se equivoca. Finalmente, al salir del ciclo de autenticación, una estricta validación condicional comprueba si se han terminado los intentos permitidos. Si el sistema ha sido vulnerado, se muestra el mensaje "Acceso denegado" de manera controlada; si no, se evita que aparezcan falsos positivos al cerrar voluntariamente el programa.

```
    else:
            control2 += 1
            print ("Credenciales incorrectas")
            print ("Intentos: ",control2," de: ", control, " Intente de nuevo: ")
if control2 >= control:
    print ("Intentos: ", control2," de: ",control, " Acceso denegado")
    inicio = False
```

Manual de uso del Módulo Principal

Al ejecutar el programa, el sistema solicitará credenciales de acceso antes de desplegar cualquier información del inventario.

1. **Usuario por defecto:** `admin`
2. **Contraseña por defecto:** `Acces2610`
3. **Control de Intentos:**
    - Dispones de un máximo de **3 intentos**.
    - Si se introducen credenciales incorrectas tres veces consecutivas, el sistema bloqueará el acceso por seguridad y finalizará la ejecución.

a) Navegación por el Menú Principal

Una vez superado el login con éxito, se desplegará el menú interactivo con las siguientes opciones numéricas:

1 - Listar productos (Inventario): Muestra en formato de tabla alineada los códigos, nombres, precios y cantidades actuales almacenados en el sistema.

2 - Agregar nuevo producto: Permite registrar un artículo nuevo.

3 - Buscar producto: Permite localizar un registro específico dentro de las listas.

4 - Eliminar producto: Remueve un registro del inventario de forma segura.

5 - Cálculos estadísticos: Ejecuta operaciones analíticas como totales, promedios y máximos/mínimos.

6 - Ordenar productos por precio: Aplica el algoritmo de ordenamiento de burbuja sobre los elementos.

7 - Salir del sistema: Finaliza el ciclo principal y cierra la aplicación de manera ordenada.

b) Manejo de Errores y Seguridad en la Entrada

El sistema cuenta con protección contra ingresos incorrectos (*defensive programming*):

- **Validación de opciones:** Si ingresas una letra, un carácter especial o un número fuera del rango (por ejemplo, `9`), el programa te advertirá con un mensaje en pantalla sin romperse ni cerrarse abruptamente.
- **Tipos de datos:** Las excepciones de tipo (`ValueError`) son capturadas automáticamente para garantizar una experiencia de usuario fluida y estable

Reflexión del Login Actual y Cómo Mejorarlo

Riesgos Identificados

- **Credenciales en texto plano (*Plain Text Hardcoding*):**
  Dado que el usuario y la contraseña están incluidos directamente en el código fuente. Si una persona tiene acceso al archivo .py, puede leer la contraseña de inmediato sin la necesidad de realizar ningún esfuerzo para descifrarla.
- **Falta de encriptación o *Hashing*:**
  Se realiza una comparación directa de la contraseña (==). Las contraseñas nunca deben ser guardadas o comparadas directamente en situaciones reales; en su lugar, se deben usar funciones que identifiquen las credenciales correctas
- **Ausencia de registros (*Logs* de auditoría):**
  El sistema no guarda un registro histórico (en un archivo de texto o base de datos) de quién intentó entrar, a qué hora, o cuántas veces falló, lo que dificulta detectar intentos maliciosos de intrusión.

Aspectos a Mejorar

- **Uso de Funciones Hash:**
  Para evitar la exposición de la clave real en el código, se podría emplear la librería hashlib de Python para que la contraseña introducida por el usuario se convierta en un código hash y se compare con el hash guardado.
- **Almacenamiento Externo Seguro:**
  Se pueden guardar las credenciales y la configuración a un archivo externo lo que facilita la modificación de las credenciales sin necesidad de alterar el código fuente.
- **Enmascaramiento de Contraseña:**
  Para impedir que los caracteres sean visibles en la pantalla al ingresar la contraseña en la consola, emplear una librería de Python especializada, como el getpass, en vez de un input() convencional.

INTEGRANTE 2

Módulo CRUD y Validación de Entradas

Este componente constituye el núcleo operativo y de control del sistema de inventario. Su diseño está enfocado en garantizar una manipulación fluida, precisa y segura de los registros, integrando una lógica robusta que protege la aplicación frente a fallos imprevistos derivados de la interacción humana.

1. Gestión Integral de Registros (Funciones CRUD)

- **Alta de Elementos (Agregar):** Mecanismo interactivo encargado de solicitar y capturar de manera estructurada los nuevos datos ingresados por el usuario, asegurando su correcta inserción y sincronización dentro de las listas paralelas del sistema.

```python
#REGISTRAR NUEVO PRODUCTO PIDIENDO LOS DATOS AL USUARIO
def registrar_producto():
    print ("REGISTRAR NUEVO PRODUCTO")

    codigo = input("Ingrese el código del producto: ")
    if not codigo:
        print("El código no puede estar vacío.")
        return
    if codigo in codigo_p:
        print("El código ya existe. No se puede registrar el producto.")
        return
    nombre = input("Ingrese el nombre del producto: ")
    if not nombre:
        print("El nombre no puede estar vacío.")
        return
```

- **Visualización de Inventario (Listar):** Módulo de despliegue diseñado para recorrer las estructuras de datos y presentar todos los elementos almacenados de forma ordenada, limpia y sumamente legible para el operador.
- **Consulta Específica (Buscar):** Algoritmo de localización optimizado para rastrear y extraer rápidamente la información de un producto en particular utilizando parámetros clave como su código o nombre identificador.

```python
#LOCALIZAR UN PRODUCTO EN ESPECÍFICO POR SU COIGO O NOMBRE
def buscar_P():
    print("BUSCAR PRODUCTO")

     if not codigo_p:
        print("No hay productos registrados.")
        return
    criterio = input("Ingrese el código o nombre del producto a buscar: ")

    encontrado = False
    for i in range(len(codigo_p)):
        #BUSQUEDA DE PRODUCTO POR CODIGO O NOMBRE (SE USA LOWER PARA IGNORAR MAYUSCULAS Y MINUSCULAS)
        if codigo_p[i] == criterio or nombre_p[i].lower() == criterio.lower():
            print(f"Producto encontrado: Código: {codigo_p[i]}, Nombre: {nombre_p[i]}, Precio: {precio_p[i]}, Cantidad: {cantidad_p[i]}")
            encontrado = True
            break

    if not encontrado:
        print("Producto no encontrado.")
```

- **Depuración de Registros (Eliminar):** Función orientada a remover de manera segura y controlada los registros obsoletos o erróneos de las listas, manteniendo la consistencia general de la base de datos local.

```python
#BORRAR UN REGISTRO DE LAS LISTAS
def eliminar_P():
    print("ELIMINAR PRODUCTO")
    if not codigo_p:
        print("No hay productos registrados.")
        return

    codigo_a_eliminar = input("Ingrese el código del producto a eliminar: ")
    if codigo_a_eliminar in codigo_p:
        idx = codigo_p.index(codigo_a_eliminar)
        confirmacion = input(f"¿Está seguro de que desea eliminar el producto? (s/n): ")
        if confirmacion == s or confirmacion == S or confirmacion == si or confirmacion == SI:
            codigo_p.pop(idx)
            nombre_p.pop(idx)
            precio_p.pop(idx)
            cantidad_p.pop(idx)
            print("Producto eliminado con éxito.")
        else:
            print("Eliminación cancelada.")
```

2. Blindaje del Sistema y Validación de Entradas

- **Control de Excepciones con Bloques `try-except`:** Implementación de barreras de seguridad en el código para interceptar y gestionar errores críticos de ejecución (por ejemplo, cuando se ingresan cadenas de texto o caracteres alfabéticos en campos numéricos estrictos), previniendo cierres abruptos o caídas del programa.

```python
#VALIDACIÓN DE PRECIO CON TRY-EXCEPT (TRY: INTENTAR Y COMPROBAR SI ES CORRECTO  .....  EXCEPT:SI OCURRE EL ERROR PASA x)
while True:
    try:
        precio = float(input("Ingrese el precio del producto: "))
        if precio < 0:
            print("El precio no puede ser negativo. Intente nuevamente.")
            continue
        break
    except ValueError:
        print("Precio inválido. Por favor, ingrese un número válido.")

#VALIDACION DE CANTIDAD CON TRY-EXCEPT
while True:
    try:
        cantidad = int(input("Ingrese la cantidad en stock: "))
        if cantidad < 0:
            print("La cantidad no puede ser negativa. Intente nuevamente.")
            continue
        break
    except ValueError:
        print("Cantidad inválida. Por favor, ingrese un número entero válido.")
```

- **Validación Cíclica y Estabilidad Operativa:** Uso de estructuras iterativas de control que validan permanentemente las acciones del usuario. Ante selecciones de menús inválidas, campos vacíos o valores fuera de rango, el sistema emite alertas descriptivas y permite reintentar la operación de forma continua, garantizando una experiencia de usuario robusta, tolerante a fallos y altamente confiable.

INTEGRANTE 3

Cálculos Estadísticos, Algoritmo de Ordenamiento por Burbuja y Valor Agregado

1. DESCRIPCIÓN DEL ROL Y OBJETIVOS

Como Integrante 3 del proyecto de Sistema de Gestión de Inventario, mi responsabilidad se enfoca en la analítica de datos, la implementación manual de algoritmos de ordenamiento y el diseño visual de reportes interactivos en consola. Las tareas asignadas comprenden:

- **• Módulo de Cálculos Estadísticos:** Desarrollo de la función calculos_E() para procesar las listas de datos y obtener indicadores clave del inventario.
- **• Algoritmo de Ordenamiento por Burbuja:** Implementación manual del algoritmo Bubble Sort dentro de la función ordenar_P() para organizar productos por precio o nombre sin desincronizar las listas.
- **• Valor Agregado y Presentación:** Estructuración de reportes tabulares avanzados e interacción limpia con el usuario.

2. EXPLICACIÓN TÉCNICA DE LO PROGRAMADO

A. Función de Cálculos Estadísticos (calculos_E)

Esta función analiza en tiempo real los datos cargados en el inventario y calcula los siguientes parámetros:

- **1. Conteo de Productos Distintos:** Determina cuántos elementos únicos existen en el catálogo leyendo la longitud de la lista de códigos (len(codigo_p)).
- **2. Cantidad Total de Unidades:** Suma todos los valores guardados en la lista cantidad_P para conocer el stock total.
- **3. Valor Total del Inventario ($):** Multiplica el precio por la cantidad de cada producto individualmente y los suma mediante sum(precio_P[i] * cantidad_P[i] for i in range(n)).
- **4. Precio Promedio por Producto ($):** Calcula el promedio aritmético simple de los precios del catálogo (sum(precio_P) / n).
- **5. Producto Más Caro y Más Barato:** Identifica los valores límite mediante max(precio_P) y min(precio_P) y utiliza sus índices correspondientes (.index()) para asociarlos con su nombre y precio correcto.

B. Función de Ordenamiento por Burbuja (ordenar_P)

Se diseñó e implementó manualmente el algoritmo de ordenamiento por burbuja (Bubble Sort) sin recurrir a métodos automáticos de Python (como .sort() o sorted()):

- **• Lógica Algorítmica:** Utiliza un bucle externo for i in range(n) y un bucle interno for j in range(0, n - i - 1) para comparar pares de elementos contiguos.
- **• Criterios Flexibles:** Permite elegir tres criterios: 1) Precio de menor a mayor, 2) Precio de mayor a menor, y 3) Nombre alfabéticamente (A-Z).
- **• Sincronización Paralela:** Cuando la condición se cumple, se efectúa un intercambio simultáneo en TODAS las listas paralelas (codigo_p, nombre_P, precio_P, cantidad_P) utilizando la sintaxis de tuplas de Python (a, b = b, a). Esto evita la desincronización de los datos del inventario.

3. MANUAL DE USO DEL MÓDULO

Pasos para ejecutar y probar las opciones desarrolladas en mi rol:

- **1. Consulta de Estadísticas (Opción 5):** Presione '5' en el menú principal para ejecutar calculos_E(). El sistema desplegará una tabla con el catálogo de artículos, total de stock, valorizado total en dólares, precio promedio y la identificación exacta del artículo con mayor y menor costo.
- **2. Ordenamiento por Burbuja (Opción 6):** Presione '6' en el menú principal. El programa solicitará seleccionar el criterio de ordenamiento (1: Precio Ascendente, 2: Precio Descendente, 3: Nombre A-Z). Tras la selección, ejecutará el algoritmo de burbuja y mostrará la lista de productos totalmente reorganizada.

4. CÓDIGO FUENTE COMPLETO

```python
# 1. MOSTRAR INVENTARIO

def listar_inventario():
    print(f"{'CÓDIGO':<10} | {'PRODUCTO':<15} | {'PRECIO ($)':<12} | {'CANTIDAD':<10}")
    print("-" * 55)
    for i in range(len(codigo_p)):
        print(f"{codigo_p[i]:<10} | {nombre_P[i]:<15} | ${precio_P[i]:<11.2f} | {cantidad_P[i]:<10}")
    print("-" * 55)
```

```python
# 2. CÁLCULOS ESTADÍSTICOS

def calculos_E():
    if not codigo_p:
        print("El inventario está vacío.")
        return

    total_distintos = len(codigo_p)
    cantidad_total = sum(cantidad_P)

    # Valor total del inventario (Precio * Cantidad)
    valor_total = sum(precio_P[i] * cantidad_P[i] for i in range(total_distintos))

    # Precio promedio
    precio_promedio = sum(precio_P) / total_distintos

    # Identificar máximo y mínimo
    max_precio = max(precio_P)
    min_precio = min(precio_P)

    idx_max = precio_P.index(max_precio)
    idx_min = precio_P.index(min_precio)

    print("=" * 55)
    print(f"{'CÁLCULOS ESTADÍSTICOS DEL INVENTARIO':^55}")
    print("=" * 55)
    print(f"• Productos distintos en catálogo : {total_distintos}")
    print(f"• Total de unidades en stock      : {cantidad_total}")
    print(f"• Valor total del inventario      : ${valor_total:.2f}")
    print(f"• Precio promedio por producto    : ${precio_promedio:.2f}")
    print(f"• Producto más caro               : {nombre_P[idx_max]} (${max_precio:.2f})")
```

```python
# 3. ALGORITNO DE BURBUJA

def ordenar_P():
    if not codigo_p:
        print("El inventario está vacío.")
        return

    print("\n--- ORDENAR PRODUCTOS (MÉTODO BURBUJA) ---")
    print("1. Ordenar por Precio (Menor a Mayor)")
    print("2. Ordenar por Precio (Mayor a Menor)")
    print("3. Ordenar alfabéticamente por Nombre (A - Z)")

    opcion = input("Seleccione una opción de ordenamiento (1-3): ")
    n = len(codigo_p)

    # Algoritmo de Burbuja manual
    for i in range(n):
        for j in range(0, n - i - 1):
            if opcion == '1':
                condicion = precio_P[j] > precio_P[j+1]
            elif opcion == '2':
                condicion = precio_P[j] < precio_P[j+1]
            elif opcion == '3':
                condicion = nombre_P[j] > nombre_P[j+1]
            else:
                print("Opción inválida.")
                return

            if condicion:
                # Intercambio simultáneo para mantener el orden de las listas
                precio_P[j], precio_P[j+1] = precio_P[j+1], precio_P[j]
                codigo_p[j], codigo_p[j+1] = codigo_p[j+1], codigo_p[j]
                nombre_P[j], nombre_P[j+1] = nombre_P[j+1], nombre_P[j]
                cantidad_P[j], cantidad_P[j+1] = cantidad_P[j+1], cantidad_P[j]

    print("\n¡Productos ordenados exitosamente con Burbuja!")
    listar_inventario()

```

PRUEBA DE EJECUCIÓN

```python

if __name__ == "__main__":
    print(">>> 1. MOSTRAR INVENTARIO ORIGINAL <<<")
    listar_inventario()

    print("\n>>> 2. PROBANDO CÁLCULOS ESTADÍSTICOS <<<")
    calculos_E()

    print("\n>>> 3. PROBANDO ORDENAMIENTO BURBUJA <<<")
    ordenar_P()
```